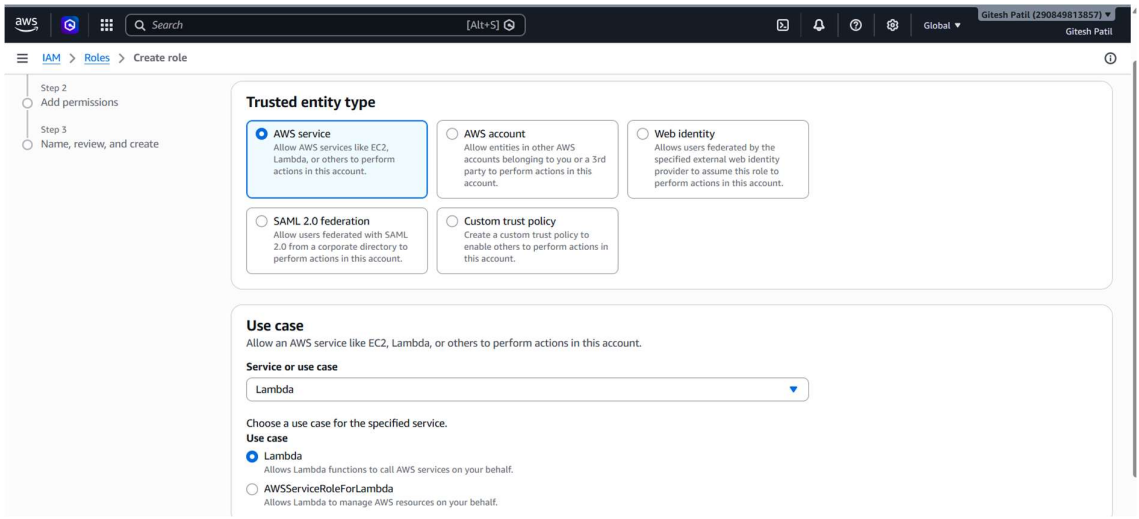
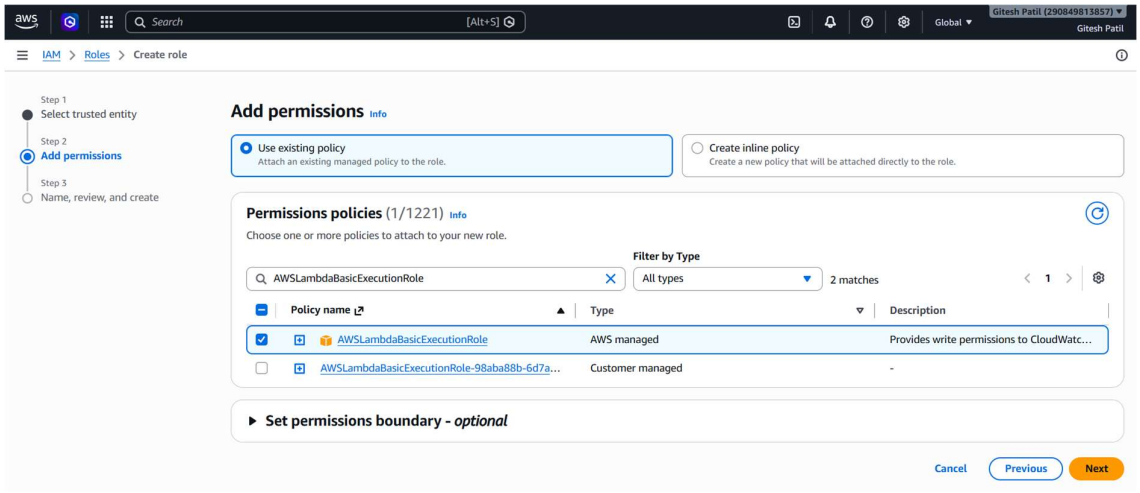
Lab 15: Build Lambda + API Gateway REST API

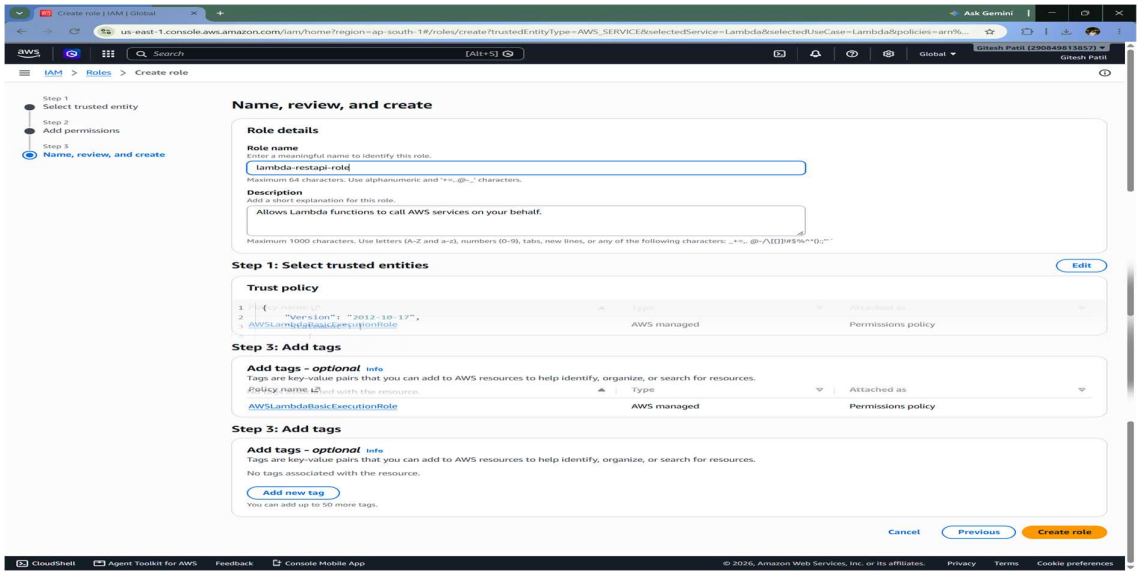
STEPS

Step 1: Create IAM Role for Lambda

1. Go to IAM Console > Roles > Create role
2. Choose:
    - Trusted Entity: AWS Service
    - Use case: Lambda
3. Click next
4. Attach Policy:
    - AWSLambdaBasicExecutionRole -> Click Next
5. Name the role: lambda-restapi-role
6. Click Create role







Step 2: Create a Lambda Function

1. Go to AWS Console -> Lambda > Create function
2. Choose Author from scratch

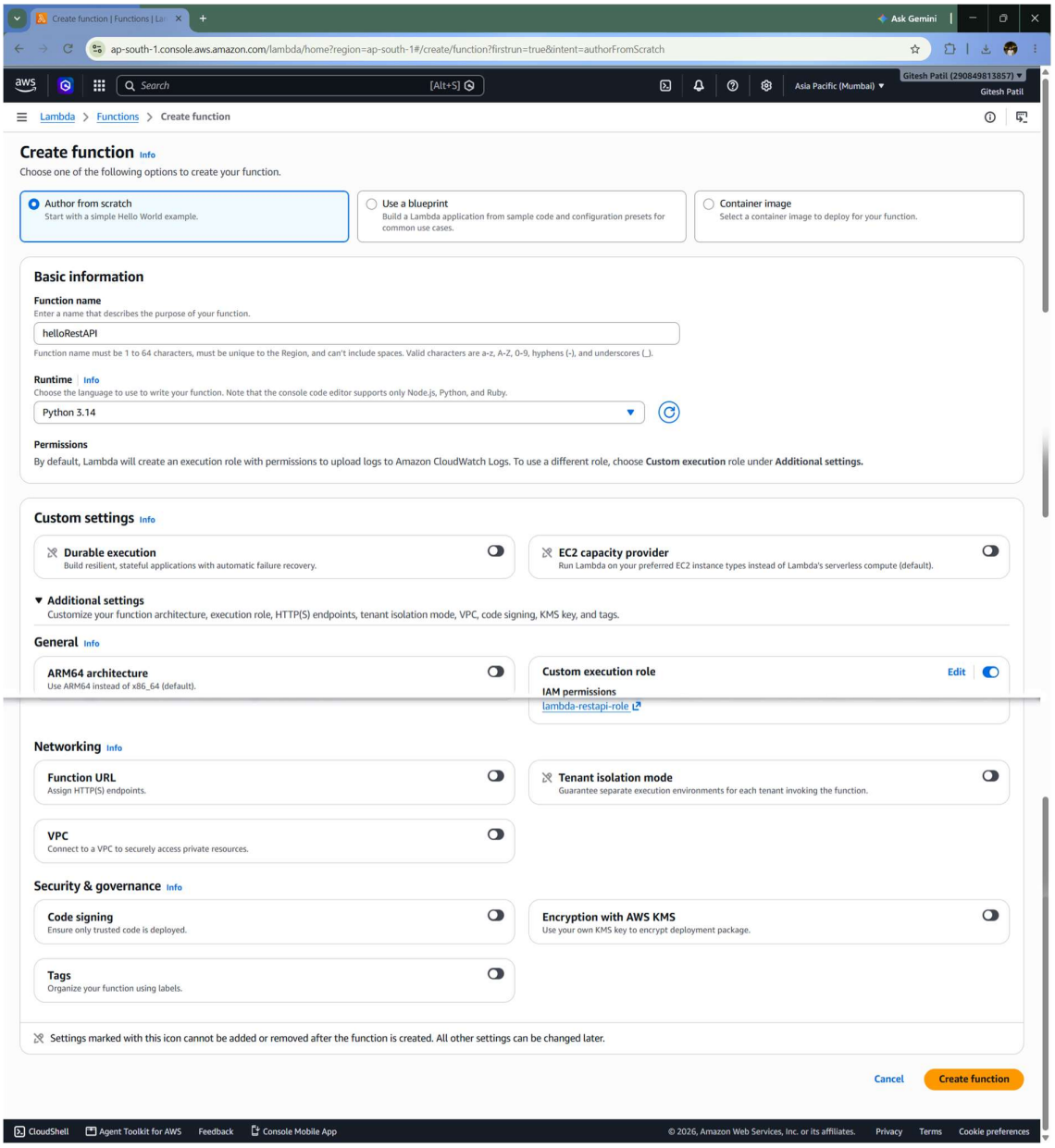
   Settings:

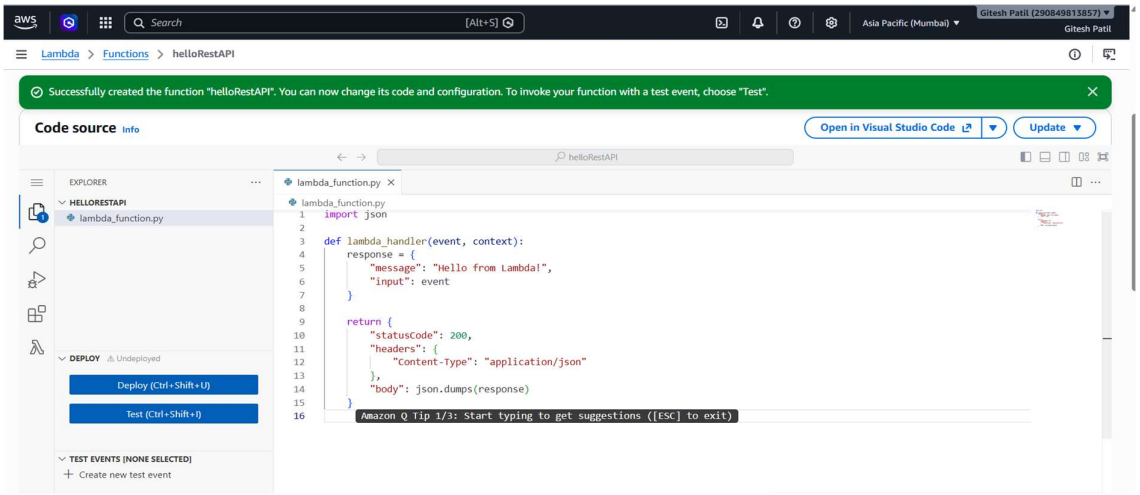
   - Function name: helloRestAPI
   - Runtime: Python 3.x
   - Execution role: choose lambda-restapi-role

3. Click Create Function
4. Add Lambda Code (Python Example):

```
import json
def lambda_handler(event, context):
    response = {
        "message": "Hello from Lambda!",
        "input": event }
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps(response)}
```

5. Deploy

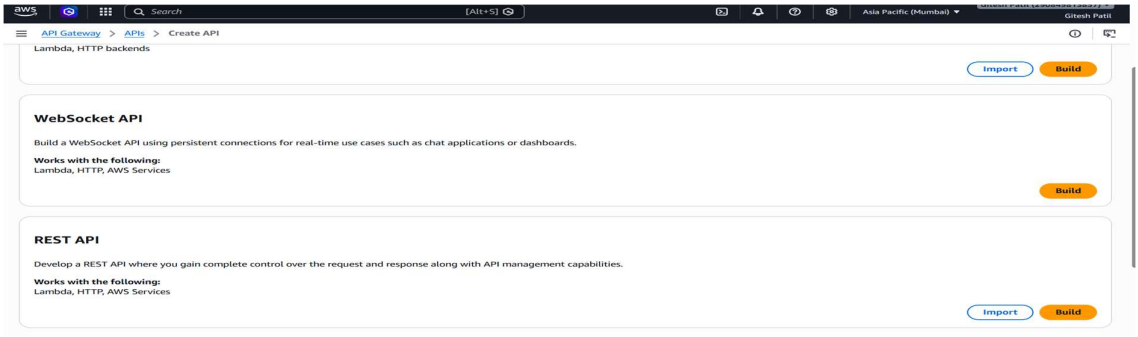

Create function | Functions | La
Ask Gemini
ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/create/function?firstrun=true&intent=authorFromScratch
Search
[Alt+S]
Asia Pacific (Mumbai)
Gitesh Patil (290849813857)
Gitesh Patil
Lambda > Functions > Create function
Create function Info
Choose one of the following options to create your function.
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
helloRestAPI
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.14
Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose Custom execution role under Additional settings.
Custom settings Info
Durable execution
Build resilient, stateful applications with automatic failure recovery.
EC2 capacity provider
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.
General Info
ARM64 architecture
Use ARM64 instead of x86_64 (default).
Custom execution role
Edit
IAM permissions
lambda-restapi-role
Networking Info
Function URL
Assign HTTP(S) endpoints.
Tenant isolation mode
Guarantee separate execution environments for each tenant invoking the function.
VPC
Connect to a VPC to securely access private resources.
Security & governance Info
Code signing
Ensure only trusted code is deployed.
Encryption with AWS KMS
Use your own KMS key to encrypt deployment package.
Tags
Organize your function using labels.
Settings marked with this icon cannot be added or removed after the function is created. All other settings can be changed later.
Cancel
Create function
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

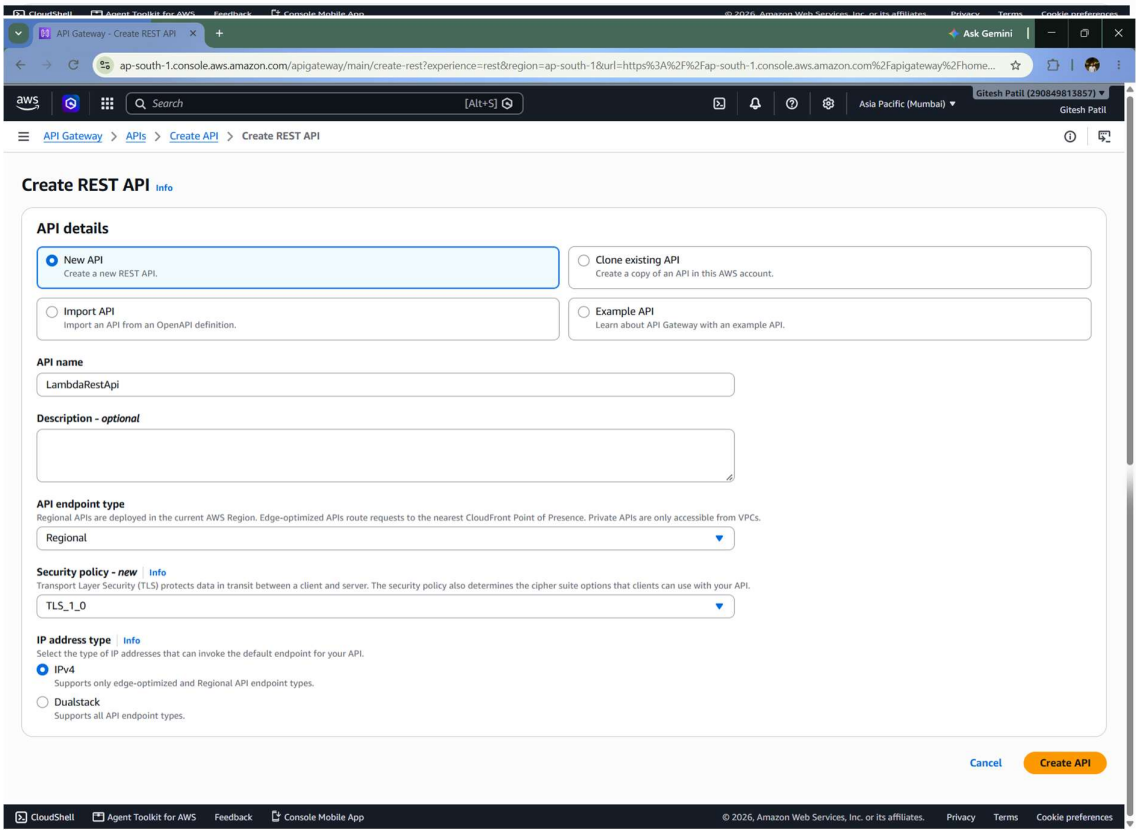

Search
[Alt+S]
Asia Pacific (Mumbai)
Gitesh Patil (290849813857)
Gitesh Patil
Lambda > Functions > helloRestAPI
Successfully created the function "helloRestAPI". You can now change its code and configuration. To invoke your function with a test event, choose "Test".
Code source Info
Open in Visual Studio Code
Update
helloRestAPI
EXPLORER
HELLORESTAPI
lambda_function.py
DEPLOY
Undeployed
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [NONE SELECTED]
Create new test event
lambda_function.py
lambda_function.py
import json
def lambda_handler(event, context):
response = {
"message": "Hello from Lambda!",
"input": event
}
return {
"statusCode": 200,
"headers": {
"Content-Type": "application/json"
},
"body": json.dumps(response)
}
Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

Step 3: Create API Gateway REST API

1. Go to API Gateway Console
2. Select Create API
3. Under REST API, choose: Build
4. Click Create API
5. Configure the API:
   - API Details: New API
   - API Name: LambdaRestApi
   - API Endpoint Type: Regional
   - Security Policy: TLS_1_0

6 Click Create API





Step 4: Create a Resource

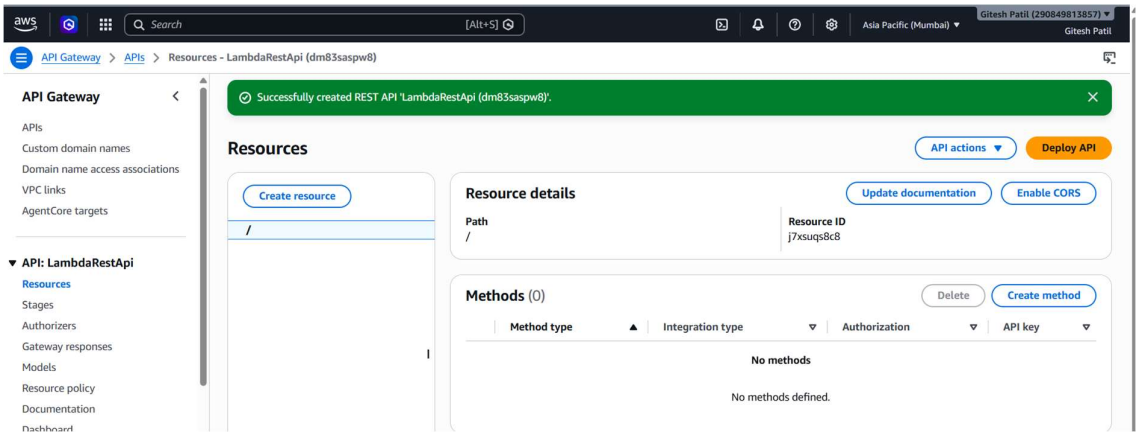
1. In the left panel, select your API

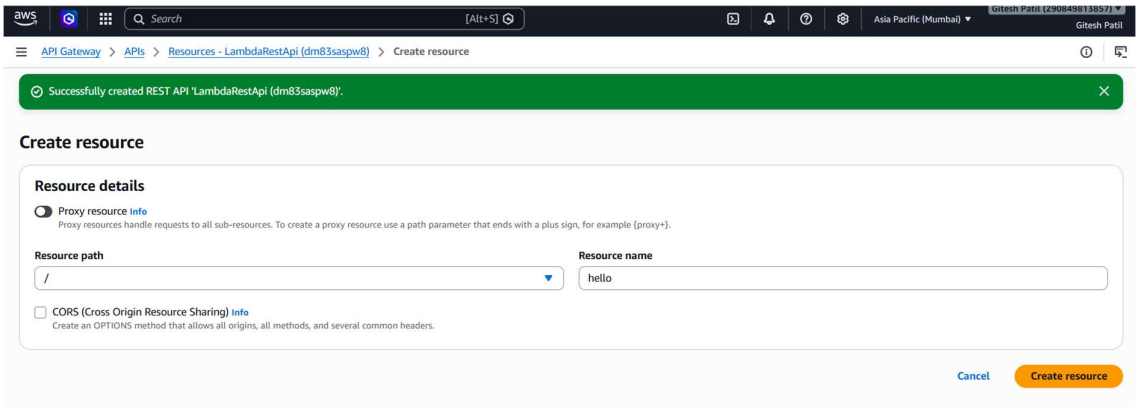
2. Click Actions > Create Resource

Settings:

- Resource Name: hello
- Resource Path: /

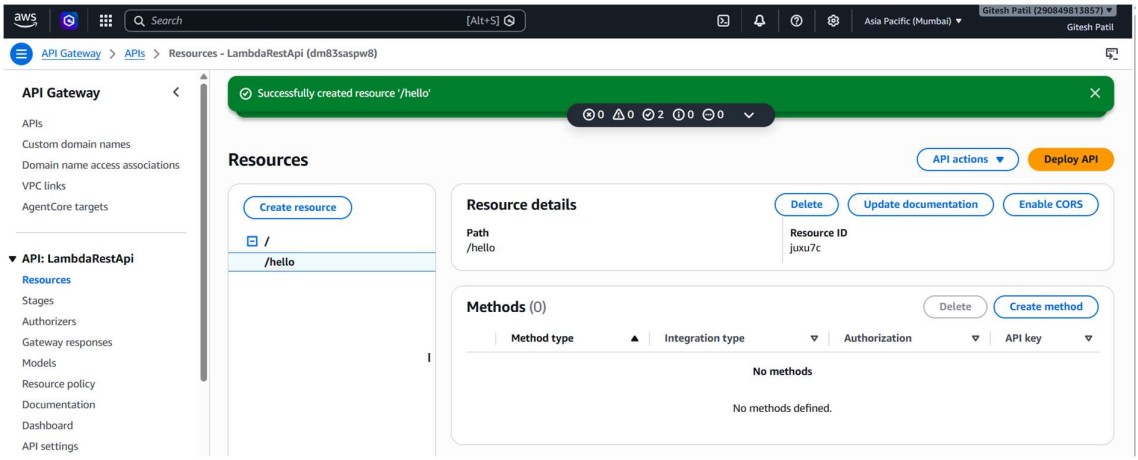
3. Click Create Resource

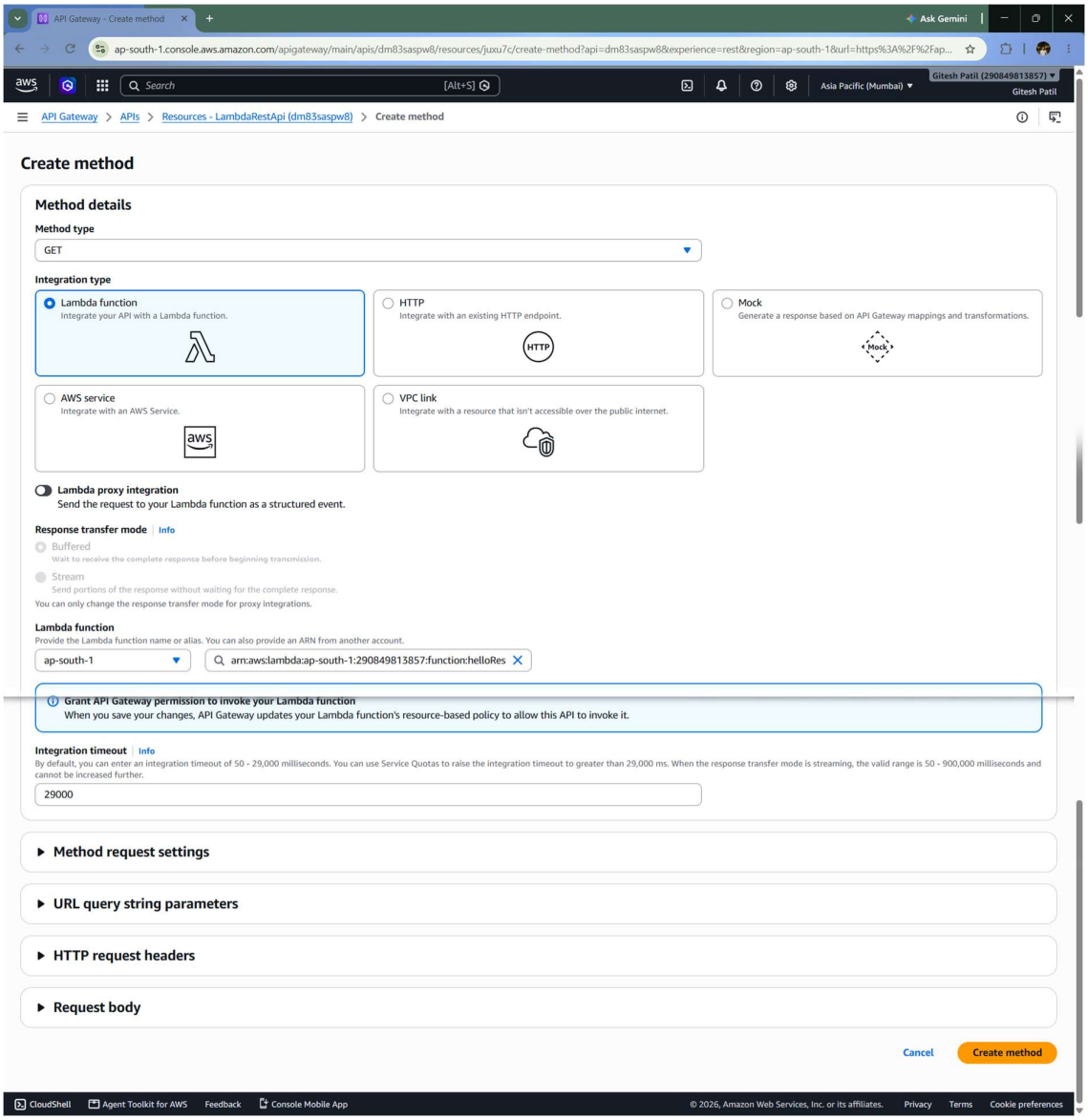


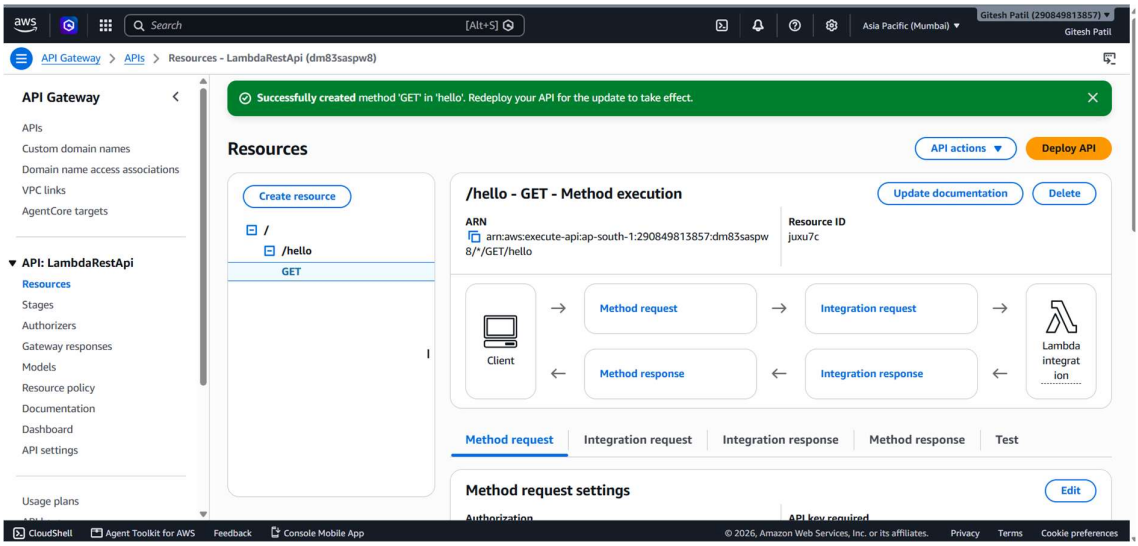


Step 5: Create GET Method Under /hello

1. Select the /hello resource
2. Click Actions > Create Method
3. Choose GET, click (Tick)
4. Configure Integration:
    - Integration type: Lambda Function
    - Region: choose your Lambda region
    - Lambda Function: helloRestAPI
    - Click Save
    - You will be asked to allow API Gateway permission to invoke your Lambda >click OK


Search
[Alt+S]
Asia Pacific (Mumbai)
Gitesh Patil (290849813857)
Gitesh Patil
API Gateway > APIs > Resources - LambdaRestApi (dm83saspw8)
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets
API: LambdaRestApi
Resources
Stages
Authorizers
Gateway responses
Models
Resource policy
Documentation
Dashboard
API settings
Successfully created resource '/hello'
Resources
API actions
Deploy API
Create resource
/
/hello
Resource details
Delete
Update documentation
Enable CORS
Path
/hello
Resource ID
juxu7c
Methods (0)
Delete
Create method
Method type
Integration type
Authorization
API key
No methods
No methods defined.


API Gateway - Create method
Ask Gemini
ap-south-1.console.aws.amazon.com/apigateway/main/apis/dm83saspw8/resources/juxu7c/create-method?api=dm83saspw8&experience=rest&region=ap-south-1&url=https%3A%2F%2Fap...
Search
[Alt+S]
Asia Pacific (Mumbai)
Gitesh Patil (290849813857)
Gitesh Patil
API Gateway > APIs > Resources - LambdaRestApi (dm83saspw8) > Create method
Create method
Method details
Method type
GET
Integration type
Lambda function
Integrate your API with a Lambda function.
HTTP
Integrate with an existing HTTP endpoint.
Mock
Generate a response based on API Gateway mappings and transformations.
AWS service
Integrate with an AWS Service.
VPC link
Integrate with a resource that isn't accessible over the public internet.
Lambda proxy integration
Send the request to your Lambda function as a structured event.
Response transfer mode | Info
Buffered
Wait to receive the complete response before beginning transmission.
Stream
Send portions of the response without waiting for the complete response.
You can only change the response transfer mode for proxy integrations.
Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.
ap-south-1
arn:aws:lambda:ap-south-1:290849813857:function:helloRes
Grant API Gateway permission to invoke your Lambda function
When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.
Integration timeout | Info
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms. When the response transfer mode is streaming, the valid range is 50 - 900,000 milliseconds and cannot be increased further.
29000
Method request settings
URL query string parameters
HTTP request headers
Request body
Cancel
Create method
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Step 6: TEST Method Within API Gateway

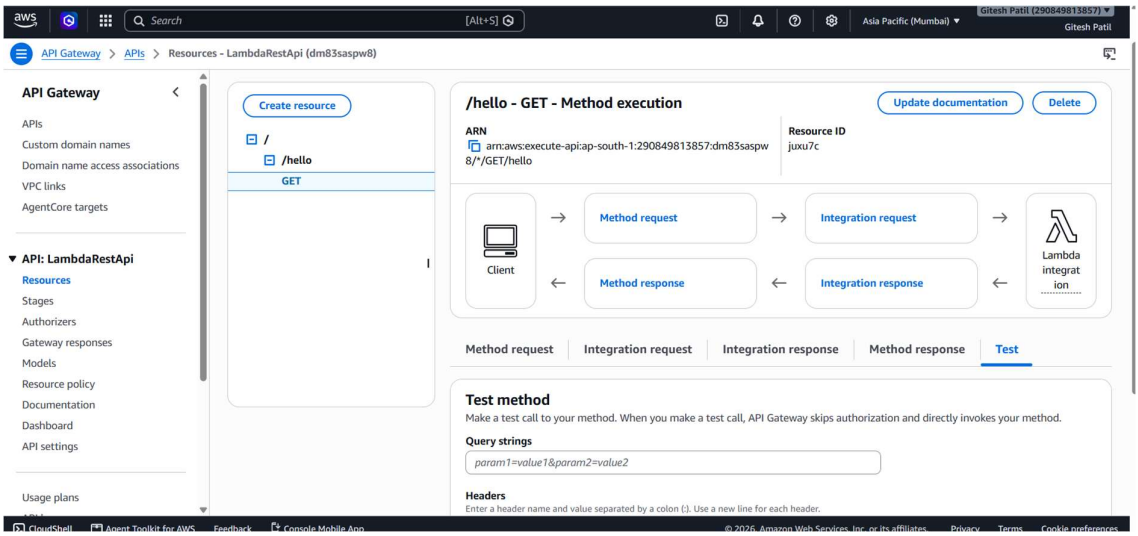
1. Select the GET method under /hello
2. Click Test
3. Click Test again (no parameters needed)

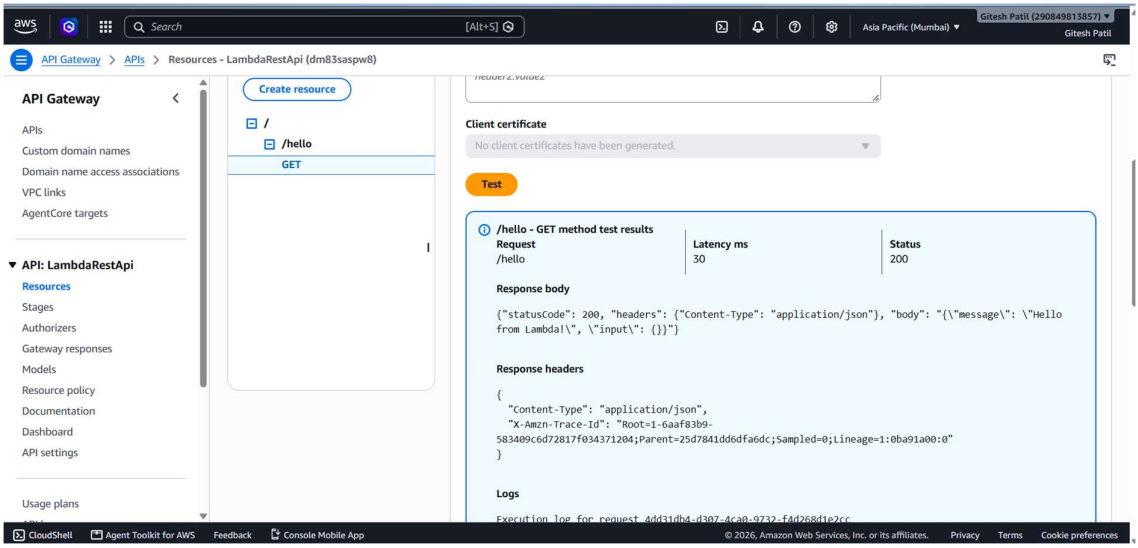
{

"message": "Hello from Lambda!",

"input": { …event data … }

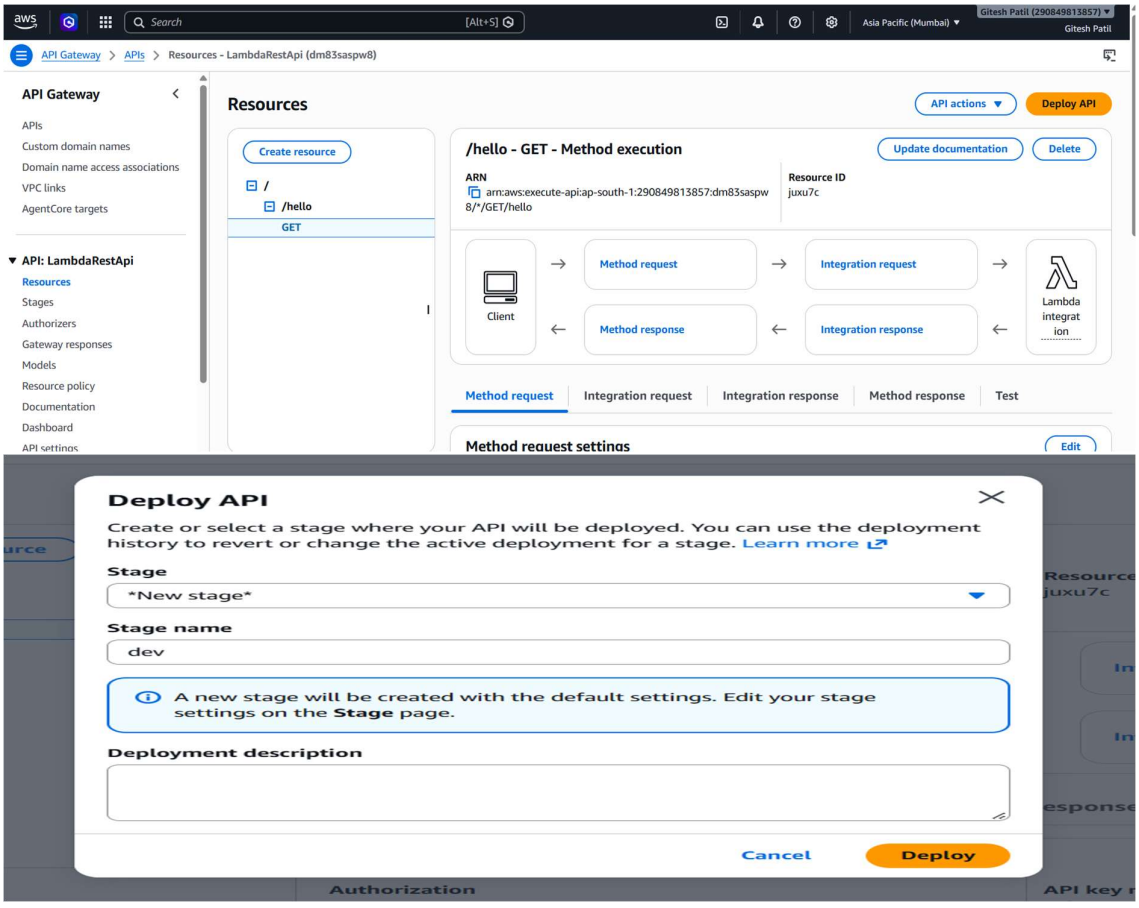
}





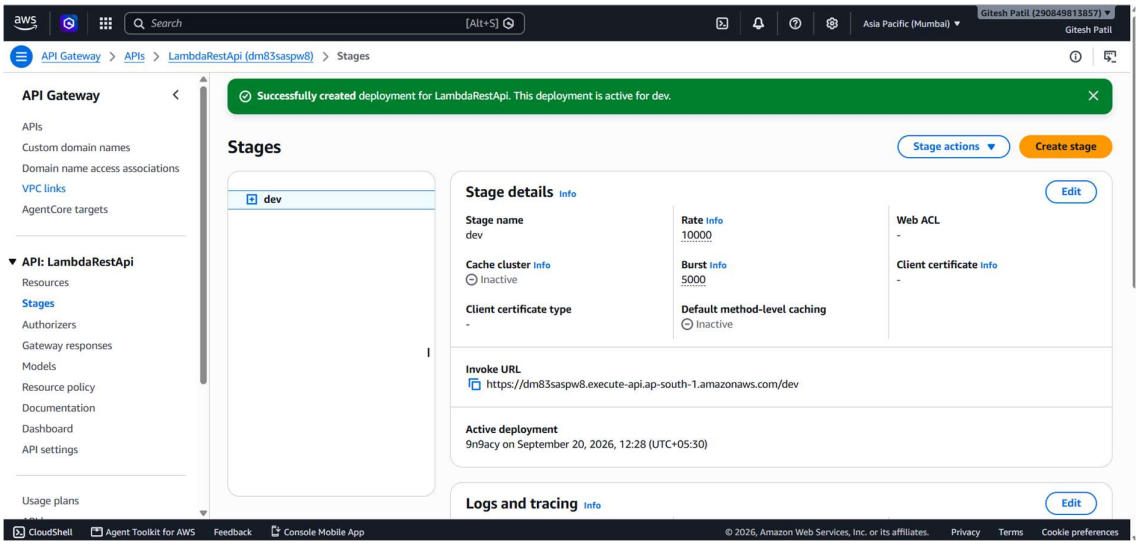
Step 7: Deploy the REST API

1. In the left panel, click Actions > Deploy API
2. Create a new stage: Stage name: dev
3. Click Deploy



4. After deployment you'll get:

- Invoke URL: https://xxxxxxxx.execute-api .< region>.amazonaws.com/dev
- Your resource will be: /hello
- So the full API endpoint is: https://xxxxxxxx.execute-api .<region>.amazonaws.com/dev/hello



Step 8: Test Your REST API (Browser or Postman)

- https://xxxxxxxx.execute-api .< region>.amazonaws.com/dev/hello
- output:

{ "message": "Hello from Lambda!",

"input": { … } }